informe-tema-plan-anti-picos-20261014

- INFORME DE TEMA: Plan Anti-Picos de Gasto
  - Dónde se va REALMENTE el dinero y cómo cortarlo
  - ⚠️ PRIMERO: LA VERDAD (corrección)
  - 1. DIAGNÓSTICO REAL — Gasto acumulado: **$12.76** en 64 sesiones
  - 2. PLAN ANTI-PICOS (6 palancas, por impacto)
  - 3. MONITOR DE ALERTA TEMPRANA (entregado)
  - 4. SI ALGÚN DÍA USAS UN MODELO CON PICOS REALES
  - 5. ACCIÓN RECOMENDADA (resumen)
  - 6. FUENTES

INFORME DE TEMA: Plan Anti-Picos de Gasto

Dónde se va REALMENTE el dinero y cómo cortarlo

Fecha: 2026-10-14 · **Clasificación:** Uso Interno — Seawolf Studio **Elaborado por:** Bellion (Gran Comandante) · **Encargado por:** Monarca (Key) **Fuente:** `state.db` de Hermes (64 sesiones del perfil default) + OpenRouter API.

⚠️ PRIMERO: LA VERDAD (corrección)

Los "picos" de precio NO son tu problema — no existen en tu modelo. `deepseek/deepseek-v4.1-flash` tiene **precio plano 24/7**: $0.01584/M input, $0.891/M output, $0.0054/M cache-read. Sin horas pico.

El "precio pico 2×" que reporté antes era de **otro** modelo (`deepseek/deepseek-v4-pro-0813`). Fue error mío y ya está corregido.

Entonces, ¿dónde se va el dinero? En **volumen de sesiones maratónicas.** Aquí los números duros.

---

1. DIAGNÓSTICO REAL — Gasto acumulado: **$12.76** en 64 sesiones

1.1 Componentes del gasto

Componente	Tokens	Precio	Costo aprox.	Peso
Cache-read	677.861.289	$0.0054/M	**$3.66**	29%
Output + razonamiento	3.79M + 1.20M	$0.891/M	**~$4.45**	35%
Input (no cacheado)	87.503.974	$0.01584/M	**$1.39**	11%
Resto (razonamiento separado, etc.)	—	—	~$3.3	25%

→ **Dos tercios del gasto = output/razonamiento + cache-read.** Ambos crecen con **sesiones largas**.

1.2 Las sesiones que se comieron el dinero (TODAS maratónicas)

Costo	Fecha	Msgs	Cache-read	Título
$2.41	2026-10-07	—	79.6M	Seawolf agent capa 2
$2.31	2026-09-30	—	**133.3M**	seawolf muse
$2.00	2026-08-15	—	52.1M	Saludo al comandante
$1.84	2026-10-10	—	94.2M	wonders smart web
$1.31	2026-09-16	—	56.6M	Dropi propio
$1.25	2026-10-04	—	41.2M	Seawolf agent capa 1

→ **Patrón claro:** cada sesión gigante (cien+ mensajes) acumula **40–133 M de cache-read** porque **cada turno reenvía el contexto completo**. La caché lo abarata (por eso $2 y no $50), pero el **volumen** manda.

1.3 Por día

- Días con sesión maratónica: **$1.25 – $2.44**.
- Días normales: **$0.00 – $0.04**.

→ El gasto **no es constante**: se concentra en días con sesiones de investigación/código largas. **Esos son "los picos" reales.**

---

2. PLAN ANTI-PICOS (6 palancas, por impacto)

#	Palanca	Ahorro estimado	Estado
1	**Cerrar sesión al pasar ~100 msgs o ~$0.50** → diario → sesión nueva	**El mayor** (corta el 70–80% del reenvío de contexto)	Protocolo listo; **aplicar disciplina**
2	**Bajar `reasoning_effort`** en turnos rutinarios (el razonamiento se cobra a $0.891/M = 35% del gasto)	Alto	Por aplicar (config de razonamiento)
3	`caveman` (output −40–65%)	Medio-alto	✅ Activo
4	`compression.threshold: 0.4` (comprime antes → menos contexto por turno)	Medio	✅ Aplicado
5	**Modelo según tarea** (rutina → `deepseek-v4-flash-0731` o gratis)	Medio	Recomendado
6	**Monitor de alerta temprana** (script)	Previene picos	✅ Entregado

Detalle de la palanca #2 (la que falta)

El **razonamiento** (1.199.274 tokens en total) se cobra a precio de output. En tareas simples no hace falta "pensar mucho". Ajuste sugerido en config de razonamiento: `reasoning_effort: low` (o `minimal`) por defecto, `high` solo para tareas críticas.

Detalle de la palanca #5

Tarea	Modelo
Investigación/código complejo	`deepseek/deepseek-v4.1-flash` (actual)
Rutina, resúmenes, clasificación	`deepseek/deepseek-v4-flash-0731` ($0.64/M out) o modelos **gratis** (`:free`)
Auxiliares/visión de fondo	ya configurado en **gratis** (freellmapi)

---

3. MONITOR DE ALERTA TEMPRANA (entregado)

Script: `intel/monitor-gasto-tokens.py` — lee `state.db` y reporta: - Gasto de hoy (UTC) y de los últimos 7 días. - Top 5 sesiones más caras. - **ALERTAS:** avisa si el día supera $2.00 o si una sesión supera $0.50.

```
python "C:\Users\Admin\intel\monitor-gasto-tokens.py"
python "C:\Users\Admin\intel\monitor-gasto-tokens.py" --umbral-sesion 0.40 --umbral-dia 1.50
```

→ Se puede programar (cron de Hermes / Tarea de Windows) para correr 1× al día y avisar.

---

4. SI ALGÚN DÍA USAS UN MODELO CON PICOS REALES

Modelos como `deepseek/deepseek-v4-pro-0813` **sí** tienen picos (2×) **L–V 01:00–04:00 y 06:00–10:00 UTC**. En tu hora (Colombia, UTC−5) eso es: | Ventana pico (UTC) | Hora Colombia | |————————|————————| | 01:00 – 04:00 | **20:00 – 23:00** | | 06:00 – 10:00 | **01:00 – 05:00** | → Si migras a un modelo así: **evita trabajar entre 20:00–23:00 y 01:00–05:00**. **Hoy NO aplica** (tu modelo es plano).

---

5. ACCIÓN RECOMENDADA (resumen)

1. **Disciplina de sesión:** si una sesión pasa de ~100 mensajes o ~$0.50 → protocolo (diario → nueva). **(palanca #1)**
2. **Aplicar** `reasoning_effort: low` por defecto. **(palanca #2)**
3. **Correr el monitor** 1×/día para detectar picos a tiempo. **(palanca #6)**
4. `caveman` + `compression 0.4` **ya activos**.

---

6. FUENTES

- `state.db` de Hermes — 64 sesiones, gasto real $12.761 (2026-10-14).
- OpenRouter API (precios/overrides por modelo) — 2026-10-14.

Fin del informe de tema.